

TÉCNICAS AVANÇADAS DE PROGRAMAÇÃO WEB E MOBILE

PROJETO PRÁTICO – DESENVOLVIMENTO DE APLICAÇÃO MULTIPLATAFORMA

API REST Pública, Persistência Local e TDD

Duração do projeto

8 semanas – 2 aulas por semana

Modalidade

Projeto em equipe

1. APRESENTAÇÃO DO PROJETO

O desenvolvimento de aplicações Web e Mobile exige muito mais do que a criação de telas.

Uma aplicação moderna precisa ser capaz de:

- apresentar uma interface adequada ao usuário;
- consumir informações disponibilizadas por servidores;
- trabalhar com APIs REST;
- interpretar dados no formato JSON;
- armazenar informações localmente;
- lidar com situações de indisponibilidade de conexão;
- organizar adequadamente seu código;
- realizar testes;
- evoluir a aplicação de forma segura.

Neste projeto, sua equipe deverá desenvolver uma **aplicação multiplataforma**, utilizando uma **API REST pública**, integrando os conhecimentos desenvolvidos na disciplina.

A proposta será desenvolvida de forma **autônoma**.

Isso significa que não será fornecido um tutorial passo a passo para construção da aplicação.

A equipe deverá pesquisar, experimentar, testar, tomar decisões técnicas e justificar as escolhas realizadas.

2. O DESAFIO

Sua equipe deverá criar um aplicativo multiplataforma que utilize dados disponibilizados por uma API REST pública para resolver um problema ou atender a uma necessidade de determinado público.

A aplicação deverá apresentar uma interface adequada para dispositivos móveis e deverá integrar:

INTERFACE + API REST + PERSISTÊNCIA LOCAL + TRATAMENTO DE ERROS + TDD

O aplicativo deverá ser mais do que uma simples tela que apresenta dados da API.

A equipe deverá transformar os dados disponíveis em uma **solução com propósito definido**.

3. API REST UTILIZADA

Neste projeto será utilizada a API pública:

DummyJSON

Documentação oficial:

<https://dummyjson.com/docs>

A API disponibiliza diferentes recursos que podem ser utilizados pelas equipes, tais como:

- produtos;
- usuários;
- carrinhos;
- posts;
- comentários;
- tarefas;
- receitas.

A documentação deverá ser consultada durante todo o desenvolvimento.

A equipe será responsável por identificar os endpoints necessários para sua aplicação.

4. ESCOLHA DO PROJETO

Cada equipe poderá escolher uma proposta de aplicação.

As sugestões abaixo servem apenas como referência.

A equipe poderá desenvolver uma solução diferente, desde que:

1. utilize a API proposta;
2. tenha um objetivo claramente definido;
3. utilize dados remotos;
4. implemente persistência local;
5. possua interface multiplataforma;
6. implemente testes;
7. seja aprovada pelo professor.

4.1 Exemplos de projetos

Aplicativo de catálogo de produtos

Possíveis funcionalidades:

- listar produtos;
- pesquisar;
- filtrar;
- ordenar;
- visualizar detalhes;
- favoritar;
- armazenar favoritos localmente.

Aplicativo de tarefas

Possíveis funcionalidades:

- listar tarefas;
- filtrar tarefas;
- criar tarefas;
- alterar tarefas;
- excluir tarefas;
- armazenar tarefas localmente;
- sincronizar informações.

Aplicativo de receitas

Possíveis funcionalidades:

- listar receitas;
- pesquisar;
- filtrar;
- visualizar detalhes;
- favoritar;

- armazenar favoritos;
- histórico de consultas.

Aplicativo de publicações

Possíveis funcionalidades:

- listar publicações;
- visualizar detalhes;
- consultar comentários;
- pesquisar;
- criar publicação;
- alterar publicação;
- excluir publicação.

Aplicativo de compras

Possíveis funcionalidades:

- catálogo;
- pesquisa;
- detalhes;
- carrinho;
- alteração de quantidade;
- cálculo de total;
- persistência local do carrinho.

5. OBJETIVO DO PROJETO

Ao final do projeto, a equipe deverá apresentar uma aplicação funcional que demonstre sua capacidade de:

- desenvolver interfaces para aplicações multiplataforma;
- consumir uma API REST;
- trabalhar com dados JSON;
- realizar requisições HTTP;
- implementar persistência local;
- tratar erros de comunicação;
- lidar com ausência de conexão;
- aplicar testes automatizados;
- utilizar princípios de TDD;
- organizar o código;
- pesquisar e solucionar problemas de forma autônoma.

6. TECNOLOGIA

A equipe deverá utilizar uma tecnologia capaz de produzir uma aplicação multiplataforma.

Exemplos:

- Ionic;
- React Native;
- Flutter;
- outra tecnologia previamente autorizada pelo professor.

A escolha deverá ser justificada.

A equipe deverá explicar:

Por que essa tecnologia foi escolhida para o desenvolvimento da solução?

7. REQUISITOS MÍNIMOS

Independentemente do projeto escolhido, a aplicação deverá possuir os requisitos apresentados a seguir.

RF01 – Tela inicial

A aplicação deverá possuir uma tela inicial que apresente as principais funcionalidades disponíveis.

RF02 – Navegação

O usuário deverá conseguir navegar entre as principais funcionalidades da aplicação.

A navegação deverá ser adequada para dispositivos móveis.

RF03 – Consumo da API REST

A aplicação deverá realizar requisições à API pública.

Deverá ser demonstrado, no mínimo:

- uma requisição GET para obtenção de dados;
- uma consulta específica por identificador;
- uma pesquisa ou filtro.

RF04 – Apresentação dos dados

Os dados recebidos da API deverão ser apresentados por meio de componentes apropriados da interface.

Exemplos:

- cards;
- listas;
- tabelas adaptadas;
- imagens;
- menus;
- botões;
- campos de pesquisa;
- filtros.

A equipe deverá preocupar-se também com a experiência de utilização.

RF05 – Persistência local

A aplicação deverá armazenar localmente pelo menos um conjunto de informações.

Exemplos:

- favoritos;
- histórico;
- carrinho;
- configurações;
- tarefas;
- últimos dados consultados.

A equipe deverá explicar:

O que será armazenado?

Por que será armazenado?

Quando será atualizado?

RF06 – Operação com ausência de conexão

A aplicação deverá apresentar um comportamento adequado quando não houver conexão com a Internet ou quando a API estiver temporariamente indisponível.

A equipe deverá definir quais funcionalidades continuarão disponíveis.

RF07 – Tratamento de erros

A aplicação deverá tratar adequadamente situações como:

- ausência de Internet;
- erro de conexão;
- erro do servidor;
- recurso inexistente;
- dados inválidos;
- falha durante o envio.

As mensagens apresentadas ao usuário deverão ser compreensíveis.

RF08 – Operação de escrita

Quando o recurso escolhido permitir, a equipe deverá implementar pelo menos uma operação de escrita.

Exemplos:

- POST;
- PUT;
- PATCH;
- DELETE.

A equipe deverá demonstrar a operação durante a apresentação.

RF09 – Indicador de carregamento

Durante operações que dependam da API, o aplicativo deverá apresentar alguma indicação de processamento.

Exemplos:

- loading;
- spinner;
- skeleton;
- mensagem de carregamento.

RF10 – TDD

A equipe deverá aplicar TDD em pelo menos **3 funcionalidades**.

Deverá ser demonstrado o ciclo:

RED → GREEN → REFACTOR

8. TDD – TEST-DRIVEN DEVELOPMENT

O projeto deverá utilizar testes como parte do processo de desenvolvimento.

Para pelo menos três funcionalidades, a equipe deverá apresentar evidências de:

RED

Teste criado e inicialmente com falha.

GREEN

Implementação realizada para fazer o teste passar.

REFACTOR

Melhoria do código mantendo os testes aprovados.

Exemplo

Funcionalidade:

O aplicativo deverá permitir pesquisar produtos pelo nome.

RED

Criar um teste que verifique o comportamento esperado.

O teste deverá falhar inicialmente.

GREEN

Implementar a funcionalidade.

O teste deverá passar.

REFACTOR

Melhorar o código sem alterar o comportamento esperado.

Os testes deverão continuar passando.

9. ARQUITETURA DA APLICAÇÃO

A equipe deverá organizar o código de forma que facilite sua manutenção.

Sempre que possível, deverão ser separados:

- componentes;
- telas;
- serviços;
- comunicação com API;
- modelos;
- persistência;
- testes.

A equipe deverá apresentar um pequeno diagrama ou descrição da arquitetura adotada.

10. CRONOGRAMA DO PROJETO

O projeto terá duração de **8 semanas**, considerando duas aulas semanais.

SEMANA 1 – ENTENDIMENTO DO DESAFIO

Aula 1

- apresentação do projeto;
- formação das equipes;
- apresentação da API;
- análise da documentação;
- levantamento dos recursos disponíveis.

Aula 2

Pesquisa inicial.

A equipe deverá identificar:

- possíveis problemas;
- público-alvo;
- recursos da API;
- possíveis funcionalidades.

Entrega

Registro inicial da proposta.

SEMANA 2 – DEFINIÇÃO DA SOLUÇÃO

A equipe deverá definir:

- nome do aplicativo;
- problema;
- público-alvo;
- objetivo;
- funcionalidades;
- recursos da API utilizados;
- tecnologia escolhida.

Entrega

Documento de proposta do projeto.

SEMANA 3 – PROTOTIPAÇÃO E ARQUITETURA

A equipe deverá desenvolver:

- wireframes;
- fluxo de navegação;
- estrutura das telas;
- definição dos componentes;
- arquitetura inicial;
- organização do projeto.

Entrega

Protótipo inicial + arquitetura.

SEMANA 4 – DESENVOLVIMENTO DA INTERFACE

A equipe deverá implementar:

- telas;
- navegação;
- componentes;
- estrutura visual;
- campos de pesquisa;
- menus;
- botões.

Neste momento, a aplicação ainda poderá utilizar dados simulados.

Meta

Ter uma versão navegável do aplicativo.

SEMANA 5 – API REST

A equipe deverá implementar o consumo da API.

Deverá funcionar pelo menos:

- consulta de dados;
- consulta por identificador;
- pesquisa ou filtro.

Meta

Aplicação obtendo dados reais da API.

SEMANA 6 – PERSISTÊNCIA E CONECTIVIDADE

Implementar:

- persistência local;
- armazenamento dos dados definidos pela equipe;
- tratamento de erros;
- tratamento de ausência de conexão;
- indicadores de carregamento.

Meta

Aplicação integrada e capaz de lidar com diferentes situações de conectividade.

SEMANA 7 – TDD E FINALIZAÇÃO

A equipe deverá:

- criar testes;
- aplicar TDD em pelo menos três funcionalidades;
- corrigir erros;
- refatorar o código;
- realizar testes finais;
- preparar a apresentação.

Meta

Versão final candidata à apresentação.

SEMANA 8 – APRESENTAÇÃO

Cada equipe deverá apresentar sua solução.

Tempo sugerido:

10 minutos de apresentação + até 5 minutos para perguntas.

11. ESTRUTURA DA APRESENTAÇÃO

A apresentação deverá seguir aproximadamente esta estrutura:

1. Problema – 1 minuto

Qual problema o aplicativo pretende solucionar?

2. Público-alvo – 1 minuto

Para quem o aplicativo foi desenvolvido?

3. Tecnologia e arquitetura – 1 minuto

Quais tecnologias foram utilizadas?

Como o sistema foi organizado?

4. Demonstração – 5 minutos

Demonstrar o aplicativo funcionando.

5. API, persistência e TDD – 2 minutos

Explicar:

- API utilizada;
- principais endpoints;
- persistência local;
- testes;
- aplicação do TDD.

6. Conclusão – 1 minuto

Principais resultados e dificuldades encontradas.

12. DEMONSTRAÇÃO OBRIGATÓRIA

Durante a apresentação, a equipe deverá demonstrar:

- ☐ Aplicativo funcionando
- ☐ Navegação
- ☐ Consumo da API
- ☐ Consulta de dados
- ☐ Pesquisa ou filtro
- ☐ Consulta detalhada
- ☐ Persistência local
- ☐ Tratamento de erro
- ☐ Comportamento sem conexão
- ☐ Operação de escrita, quando aplicável
- ☐ Testes automatizados
- ☐ Evidências de TDD

13. ENTREGAS

Ao final do projeto, a equipe deverá entregar:

1. Código-fonte

Projeto completo.

2. Aplicativo

Aplicação funcionando em dispositivo físico, emulador ou ambiente compatível.

3. Documentação

A documentação deverá conter:

1. identificação da equipe;
2. nome do aplicativo;
3. problema;
4. público-alvo;
5. objetivo;
6. funcionalidades;
7. tecnologia;
8. arquitetura;
9. API utilizada;
10. endpoints utilizados;
11. persistência local;
12. tratamento de erros;
13. estratégia para ausência de conexão;
14. testes;
15. TDD;
16. dificuldades encontradas;
17. soluções adotadas;
18. conclusão.

4. Apresentação

Apresentação da solução funcionando.

14. CRITÉRIOS DE AVALIAÇÃO

Critério	Pontuação
Interface e experiência de usuário	1,5
Consumo da API REST	2,0
Persistência local	1,5
Tratamento de erros e conectividade	1,0
Funcionalidades e solução do problema	1,0
TDD e testes automatizados	1,5
Organização e qualidade do código	1,0
Documentação e apresentação	0,5
TOTAL	10,0

15. AUTONOMIA E PESQUISA

Este projeto será avaliado também pela capacidade da equipe de pesquisar e solucionar problemas.

Durante o desenvolvimento, a equipe poderá utilizar:

- documentação oficial;
- livros;
- artigos;
- tutoriais;
- vídeos;
- fóruns técnicos;
- ferramentas de IA.

Entretanto, os integrantes deverão compreender o código produzido e serão responsáveis por explicar as decisões tomadas.

Durante a apresentação, o professor poderá solicitar que qualquer integrante explique:

- uma parte do código;
- uma chamada à API;
- uma estrutura de dados;
- a persistência utilizada;
- um teste;
- uma decisão arquitetural.

16. USO DE INTELIGÊNCIA ARTIFICIAL

Ferramentas de Inteligência Artificial poderão ser utilizadas como **ferramentas de apoio à pesquisa e desenvolvimento**.

Entretanto:

A utilização de IA não elimina a responsabilidade do aluno sobre o código produzido.

A equipe deverá ser capaz de explicar o funcionamento das principais partes da aplicação.

Quando a IA for utilizada de maneira significativa, recomenda-se registrar:

- ferramenta utilizada;
- finalidade;
- pergunta/prompt utilizado;

- resultado obtido;
- alterações realizadas pela equipe.

O objetivo é utilizar a IA como ferramenta de aprendizagem e desenvolvimento, e não simplesmente copiar uma solução sem compreendê-la.

17. DESAFIO ADICIONAL

Após concluir os requisitos obrigatórios, a equipe poderá implementar uma funcionalidade adicional.

Exemplos:

- modo escuro;
- notificações;
- paginação;
- cache;
- dashboard;
- autenticação;
- acessibilidade;
- animações;
- sincronização automática;
- funcionamento offline ampliado.

A funcionalidade adicional deverá apresentar uma justificativa.

A equipe deverá explicar:

Qual necessidade foi identificada e como a funcionalidade proposta melhora a solução?

18. REGRAS DO PROJETO

1. O aplicativo deverá consumir efetivamente a API REST pública.
2. Não será suficiente apresentar dados previamente copiados da API.
3. O aplicativo deverá possuir persistência local.
4. A aplicação deverá possuir tratamento de erros.
5. A equipe deverá implementar testes automatizados.
6. Pelo menos três funcionalidades deverão utilizar TDD.
7. O código deverá ser organizado.
8. Todos os integrantes deverão participar do desenvolvimento.
9. Todos os integrantes deverão conhecer a solução apresentada.
10. A equipe deverá respeitar os prazos definidos para cada etapa.

19. CHECKLIST DA EQUIPE

Antes da apresentação, verifique:

Planejamento

- ☐ Definimos o problema.
- ☐ Definimos o público-alvo.
- ☐ Definimos o objetivo.
- ☐ Definimos as funcionalidades.

Interface

- ☐ Temos uma tela inicial.
- ☐ A navegação funciona.
- ☐ Os componentes são adequados.
- ☐ A interface é adequada para dispositivos móveis.

API

- ☐ A aplicação consome a API.
- ☐ Utilizamos GET.
- ☐ Utilizamos consulta específica.
- ☐ Utilizamos pesquisa/filtro.
- ☐ Implementamos operação de escrita quando aplicável.

Persistência

- ☐ Temos dados armazenados localmente.
- ☐ Sabemos explicar por que os dados são armazenados.
- ☐ A aplicação consegue recuperar os dados locais.

Conectividade

- ☐ Tratamos ausência de conexão.
- ☐ Tratamos erros da API.
- ☐ Temos indicador de carregamento.

TDD

- ☐ Temos pelo menos três funcionalidades testadas.
- ☐ Temos evidência do RED.
- ☐ Temos evidência do GREEN.
- ☐ Temos evidência do REFACTOR.

Apresentação

- ☐ O aplicativo está funcionando.
- ☐ O código está organizado.
- ☐ Todos os integrantes sabem explicar a solução.
- ☐ A documentação está pronta.
- ☐ A apresentação está preparada.

20. QUESTÃO FINAL

Ao concluir o projeto, a equipe deverá refletir sobre a seguinte questão:

Como transformar dados disponibilizados por uma API REST pública em uma aplicação multiplataforma útil, confiável e adequada às necessidades de seus usuários?

A resposta deverá ser construída não apenas por meio da documentação, mas principalmente por meio da **solução desenvolvida pela equipe**.

RESULTADO ESPERADO

Ao final das oito semanas, espera-se que cada equipe apresente uma aplicação multiplataforma funcional capaz de demonstrar a integração dos principais conceitos estudados na disciplina:

COMPONENTES DE INTERFACE



NAVEGAÇÃO



CONSUMO DE API REST



PROCESSAMENTO DE JSON



PERSISTÊNCIA LOCAL



TRATAMENTO DE CONECTIVIDADE



TESTES AUTOMATIZADOS



TDD



APLICAÇÃO MULTIPLATAFORMA FUNCIONAL